CampusRide

System Design Document

A real-time campus e-rickshaw ride coordination platform connecting passengers, verified drivers, and administrators on university campuses.

Version 1.0 · June 2026 · Campus Mobility Platform

Contents

1. Problem Understanding

1.1 Context

Large university campuses (e.g. IIT Roorkee) require frequent short e-rickshaw trips between buildings. Informal coordination causes unpredictable waits, no trip visibility, cash-only payments, unverified drivers, and no demand data for drivers.

1.2 Solution

CampusRide digitizes the full ride lifecycle on a campus mobility marketplace:

```
Passenger requests ride → Online drivers notified (WebSocket) → Driver accepts →
Start → Complete → Pay & Rate
```

1.3 User Roles

PASSENGER

Request rides on a map, track live status, pay via UPI/QR/cash, rate drivers, and view ride history.

DRIVER

Submit verification documents, go online, accept/reject requests, manage the ride lifecycle, broadcast GPS, and view analytics.

ADMIN

Review and approve or reject driver verification submissions before drivers can go online and accept rides.

1.4 Key Features

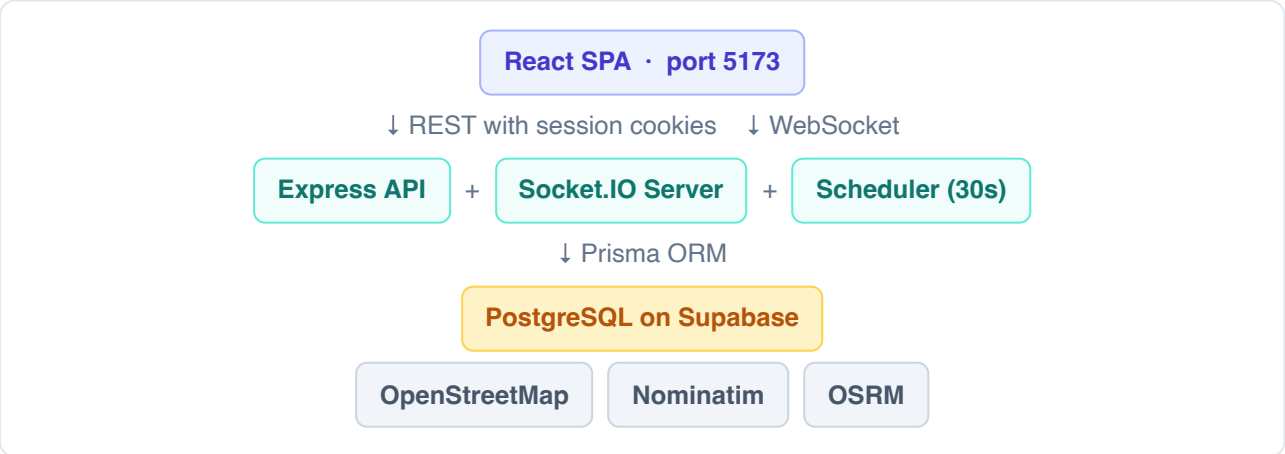
- Map-based pickup and destination selection
- Nominatim geocoding (campus-bounded search)
- Scheduled rides (15 minutes – 7 days ahead)
- Real-time ride status via Socket.IO
- Live driver GPS tracking and OSRM route polylines
- Simulated payment with transparent fare calculation
- Driver verification and admin approval workflow
- Campus demand analytics and heatmaps
- Ride history, star ratings, and written feedback
- Optional browser push notifications

2. System Architecture

2.1 Technology Stack

Layer	Technologies	Purpose
Frontend	React 19, Vite 6, TypeScript, Tailwind CSS v4, Zustand, React Router 7	Role-based single-page application (:5173)
Maps & Real-time	Leaflet, Socket.IO Client 4.8, QRCode	Interactive maps, live events, payment QR display
Backend	Node.js, Express 5, Socket.IO 4.8, Zod, bcryptjs	REST API and WebSocket server (:3001)
Data	Prisma 6.9, PostgreSQL (Supabase)	ORM and shared cloud database
Authentication	express-session + connect-pg-simple	Server-side sessions stored in Postgres
External Services	OpenStreetMap, Nominatim, OSRM	Map tiles, geocoding, route polylines (no API keys)

2.2 Component Diagram



Backend layering: Routes (HTTP + Zod validation) → Services (business logic) → Prisma (data access)

2.3 Frontend Routes

Path	Role	Pages
`/login` , `/register`	Public	Authentication flows
`/passenger`	PASSENGER	Map home, ride history, profile
`/driver`	DRIVER	Dashboard, requests, analytics, profile
`/admin`	ADMIN	Driver verification review

2.4 Real-Time Communication (Socket.IO)

Socket connections authenticate via the same session middleware as REST.

Room / Event	Description
`user:{userId}`	Per-user ride notifications
`drivers:online`	Broadcast new ride requests to online drivers
`ride:{rideId}`	Status updates and live driver location
`ride:requested`	Emitted when a new or scheduled ride becomes active
`ride:accepted` / `ride:cancelled`	Driver accepts or either party cancels
`ride:status:update`	Ride started or completed
`driver:location`	Driver GPS pushed to passenger during active ride

3. Database Schema

PostgreSQL managed via Prisma (`prisma db push`). Six application tables plus an implicit `session` table for cookie storage.

3.1 Enumerations

Enum	Values
Role	PASSENGER, DRIVER, ADMIN
RideStatus	REQUESTED → ACCEPTED → IN_PROGRESS → COMPLETED | CANCELLED
VerificationStatus	PENDING, VERIFIED, REJECTED
PaymentStatus	PENDING, COMPLETED, FAILED
PaymentMethod	UPI, QR_CODE, CASH

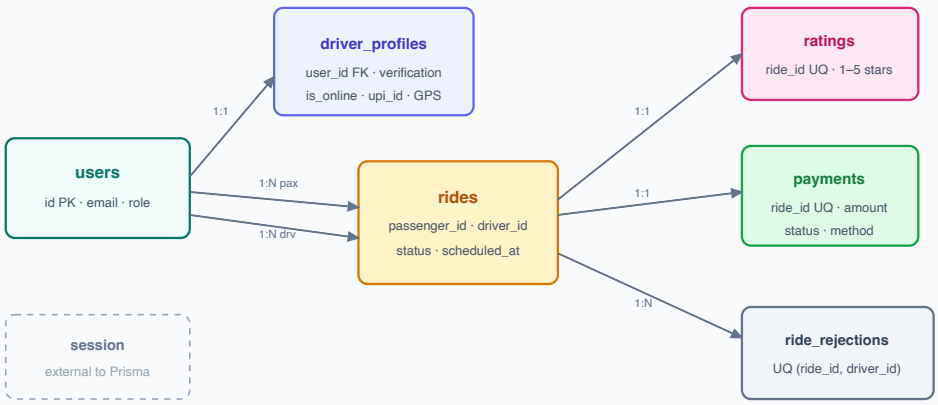
3.2 Core Tables

Table	Key Columns	Notes
users	id (cuid PK), email (unique), password_hash, name, phone?, role	Central user identity
driver_profiles	user_id (unique FK), vehicle_type/number, license & govt ID, verification_status, upi_id?, is_online, current_lat/lng	One-to-one with driver users
rides	passenger_id, driver_id?, pickup/destination text + coordinates, status, scheduled_at?, lifecycle timestamps	Indexed on status, driver, and passenger
ratings	ride_id (unique FK), passenger_id, driver_id, rating (1–5), feedback?	One rating per completed ride
ride_rejections	ride_id, driver_id, rejected_at	Unique pair (ride_id, driver_id)
payments	ride_id (unique FK), passenger_id, driver_id, amount, status, method?, upi_transaction_id?	Auto-created on ride completion
session	sid (PK), sess (JSON), expire	connect-pg-simple; 7-day cookie TTL

3.3 Ride State Machine

```
REQUESTED ──accept──▶ ACCEPTED ──start──▶ IN_PROGRESS ──complete──▶ COMPLETED
                  Any active state ──cancel──▶ CANDELLED
```

4. Entity Relationship Diagram



Relationship	Cardinality	On Delete
users ↔ driver_profiles	1 : 0..1	CASCADE
users → rides (as passenger or driver)	1 : N	—
rides → ratings / payments	1 : 0..1 each	CASCADE
rides → ride_rejections; users → ride_rejections (driver)	1 : N	CASCADE

5. API Overview

> **Base URL:** `http://localhost:3001/api` · **Auth:** session cookie `campusride.sid` · All requests use `credentials: include`

5.1 Authentication — `/api/auth`

Method	Path	Auth	Description
GET	`/health`	No	Server health check
POST	`/register`	No	Register as passenger or driver; creates session
POST	`/login`	No	Authenticate and set session cookie
POST	`/logout`	No	Destroy session and clear cookie
GET	`/me`	Yes	Return current user and driver profile
PUT	`/profile`	Yes	Update name and phone number

5.2 Rides — `/api/rides`

Method	Path	Role	Description
POST	`/`	Passenger	Create a ride (immediate or scheduled)
GET	`/active`	Any	Get the user's current active ride
GET	`/pending`	Driver	List available requests excluding rejections
GET	`/history`	Any	Last 50 completed or cancelled rides
GET	`/:id`	Any	Single ride with rating and payment details
PUT	`/:id/accept`	Driver	Accept a ride request
PUT	`/:id/reject`	Driver	Decline a ride request
PUT	`/:id/start`	Driver	Transition ACCEPTED → IN_PROGRESS
PUT	`/:id/complete`	Driver	Complete ride; auto-creates payment record
PUT	`/:id/cancel`	Passenger / Driver	Cancel ride with optional reason
POST	`/:id/rate`	Passenger	Submit 1–5 star rating and optional feedback

5.3 Drivers — `/api/drivers`

Method	Path	Description
PUT	`/location`	Update GPS coordinates; push to passenger during active ride
PUT	`/status`	Toggle online/offline (verified drivers only)
PUT	`/verification`	Submit license number and government ID for review
PUT	`/upi`	Update UPI ID for payment QR generation
PUT	`/vehicle`	Update vehicle type and registration number
GET	`/dashboard`	Driver stats, active rides, today's earnings, recent history
GET	`/analytics`	Personal stats and campus-wide demand insights
GET	`/available`	List all currently online drivers

5.4 Payments — `/api/payments` · Admin — `/api/admin`

Method	Path	Description
GET	`/api/payments/ride/:rideId`	Get payment record for a specific ride
PUT	`/api/payments/:id/complete`	Mark payment complete (UPI, QR_CODE, or CASH)
GET	`/api/payments/history`	Payment history filtered by user role
GET	`/api/admin/verifications/pending`	List drivers awaiting verification review
PUT	`/api/admin/verifications/:userId`	Approve or reject a driver verification

6. Design Decisions

Session auth over JWT — `express-session` with a Postgres store suits same-origin cookie auth and server-side logout. JWT utilities exist but are unused.

Optimistic locking — Accept/start/complete use `updateMany` with status preconditions to prevent double-accept races.

One active ride per user — Passengers: REQUESTED/ACCEPTED/IN_PROGRESS; drivers: ACCEPTED/IN_PROGRESS only.

Driver verification gate — Drivers must be VERIFIED to go online; rejection forces offline and records a reason.

Per-driver rejections — Unique (ride_id, driver_id) hides declined rides from that driver only.

Scheduled ride poller — 30s interval activates due rides and emits `ride:requested` (15 min – 7 day window).

Simulated payments — Fare: ₹10 + ₹5/km (haversine), rounded to ₹5, min ₹10; flat ₹20 without coordinates.

Targeted Socket.IO rooms — GPS pushed only during active rides; driver auto-offline on disconnect.

Client-side saved locations — Recent/saved places in `localStorage`; not synced server-side.

Campus-bounded geocoding — Nominatim viewbox around IIT Roorkee for relevant search results.

Prisma db push — Schema via `db push` on shared Supabase; no versioned migrations in repo.

Mobile-first UI — "Luminous Serenity" design: glass/aurora aesthetic, ~600px max-width, bottom sheets.

Resilience — API retry, offline banner, socket reconnect feedback, and manual refresh buttons.
